Planck Polarization Component Separation in STL

Alexandros Tsouros

Problem setup

We observe a Planck polarization patch:

$$d_Q,\ d_U \in \mathbb{R}^{H\times W}.$$

The goal is to recover two unknown signal maps

$$\tilde{s} = (\tilde{s}_Q, \tilde{s}_U),$$

using:

- the observed Q/U maps,
- paired nuisance samples $(n_Q^{(i)}, n_U^{(i)})$,
- a fixed auxiliary dust tracer d_I from I857.

What is optimized?

The optimization variable is the two-map tensor

$$u = \begin{bmatrix} u_Q \\ u_U \end{bmatrix} \in \mathbb{R}^{2\times H\times W}.$$

Everything else is fixed during a minibatch step:

$$d_Q,\ d_U,\ d_I, \quad \{(n_Q^{(i)}, n_U^{(i)})\}_{i\in I}.$$

The code initializes u either from the data maps or from white noise with matching standard deviations. After optimization, the final recovered maps are denoted

$$\tilde{s}_Q,\ \tilde{s}_U.$$

Core idea: match scattering statistics

Define a feature map $\Phi(\cdot)$ from STL scattering statistics.

For a minibatch of nuisance indices I, compare:

$$y(I) = \Phi(X_{\text{target}}(I)), \qquad \tilde{y}(u; I) = \Phi(X_{\text{run}}(u; I)).$$

The loss is simply

$$\mathcal{L}(u; I) = \|\tilde{y}(u; I) - y(I)\|_2^2.$$

Choosing noise realizations

The nuisance files are paired before optimization:

$$(n_Q^{(j)}, n_U^{(j)}) \quad \text{share the same noise/CMB-residual seed key.}$$

At one optimization step, the script samples a set of indices

$$I = \{i_1, \ldots, i_{N_b}\},$$

For those indices it loads

$$n_Q^I, \; n_U^I \in \mathbb{R}^{N_b \times H \times W}.$$

This leading dimension is the noise-realization dimension.

The batch dimension

Each selected realization is treated as one batch element. The maps that do not depend on the realization are broadcast along this dimension:

$$d_Q, d_U, d_I, u_Q, u_U \in \mathbb{R}^{H\times W} \quad \longrightarrow \quad \mathbb{R}^{N_b\times H\times W}.$$

After broadcasting fixed maps and inserting the sampled nuisance maps, the channels are stacked:

$$X_{\mathrm{target}}(I),\ X_{\mathrm{run}}(u; I) \in \mathbb{R}^{N_b\times C\times H\times W}.$$

Here $C = 5$ in the full phase and $C = 3$ in the optional warm-up phase.

Averaging over realizations

STL is applied to the whole batch tensor, not to one realization at a time:

$$X(I) \in \mathbb{R}^{N_b \times C \times H \times W} \quad \longmapsto \quad \Phi(X(I)).$$

The flattened statistics are then averaged along the batch dimension. So the loss compares the *mean* STL statistics over the selected realizations:

$$\bar{\Phi}_{\text{target}}(I) \quad \text{against} \quad \bar{\Phi}_{\text{run}}(u; I).$$

This means one optimizer step asks the same running maps u_Q, u_U to work across several nuisance realizations at once.

The target channels

For each selected nuisance pair $(n_Q^{(i)}, n_U^{(i)})$, the target batch element is

$$X_{\text{target}}^{(i)} = \begin{bmatrix} d_Q \\ n_Q^{(i)} \\ d_U \\ n_U^{(i)} \\ d_I \end{bmatrix} \in \mathbb{R}^{5\times H\times W}.$$

Across all selected indices:

$$X_{\text{target}}(I) \in \mathbb{R}^{N_b\times 5\times H\times W}.$$

The observed maps and auxiliary map are repeated across the batch; only the nuisance maps vary with i.

The running channels

For the current running maps $u = (u_Q, u_U)$, each running batch element is

$$X_{\mathrm{run}}^{(i)}(u) = \begin{bmatrix} u_Q + n_Q^{(i)} \\ d_Q - u_Q \\ u_U + n_U^{(i)} \\ d_U - u_U \\ d_I \end{bmatrix}.$$

Across all selected noise realizations:

$$X_{\mathrm{run}}(u; I) \in \mathbb{R}^{N_b \times 5 \times H \times W}.$$

Each polarization field is split into:

- a *data-like* channel: running map plus sampled nuisance,
- a *residual* channel: what remains in the observed map.

How fields are put into one channel

The first and third running channels are the key construction:

$$u_Q + n_Q^{(i)}, \qquad u_U + n_U^{(i)}.$$

This deliberately makes one channel look like an observed polarization map:

$$\text{structured signal field} + \text{one nuisance realization}.$$

The optimizer changes the same u_Q, u_U for all batch elements. The only difference between batch elements is which nuisance realization is added.

The residual channels

The second and fourth channels are not sampled:

$$r_Q = d_Q - u_Q, \qquad r_U = d_U - u_U.$$

They are repeated across the minibatch:

$$r_Q,\ r_U \in \mathbb{R}^{H\times W} \quad \longrightarrow \quad \mathbb{R}^{N_b\times H\times W}.$$

This lets STL see whether the part removed from the data behaves like the nuisance channels.

Auxiliary dust tracer

The I857 map is a fixed auxiliary channel:

$$d_I = I857 - \langle I857 \rangle.$$

It is included in both target and running tensors, unchanged:

$$X_{\text{target}} : \ d_I, \qquad X_{\text{run}} : \ d_I.$$

Its role is to give the scattering statistics access to morphology correlated with polarized dust.

Which channel relations are measured?

The 5-channel cross matrix keeps only selected statistics:

$$\begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

So STL measures:

- auto-statistics of all channels,
- Q–aux and U–aux relations,
- no explicit Q–U or signal–residual cross terms.

Optional warm-up without explicit noise channels

The script can begin with a simpler 3-channel phase:

$$X^{(i)}_{\mathrm{target}} = \begin{bmatrix} d_Q \\ d_U \\ d_I \end{bmatrix}, \qquad X^{(i)}_{\mathrm{run}}(u) = \begin{bmatrix} u_Q + n^{(i)}_Q \\ u_U + n^{(i)}_U \\ d_I \end{bmatrix}.$$

This phase matches data-like Q/U channels before introducing explicit residual channels.

Normalization and operators

Two STL operators are built:

$$\Phi_3 \quad \text{for 3 channels,} \qquad \Phi_5 \quad \text{for 5 channels.}$$

Each stores a fixed normalization reference once, then reuses it during optimization:

$$\texttt{store\_ref} \quad \rightarrow \quad \texttt{load\_ref}.$$

Minibatch loop

At each optimization step:

1. choose paired nuisance indices I,
2. build target statistics once,
3. build running statistics inside the closure,
4. take one L-BFGS optimizer step.

The target statistics are computed without gradients. The running statistics are computed inside the closure so gradients flow back to u_Q, u_U.

Takeaways

- The optimized running maps are u_Q, u_U; the final maps are denoted $\tilde{s}_Q, \tilde{s}_U$.
- Target channels contain observed Q/U, sampled Q/U nuisance, and fixed I857.
- Running channels mix fields as $u_Q + n_Q$ and $u_U + n_U$.
- Residual channels $d_Q - u_Q$ and $d_U - u_U$ expose what the running maps leave behind.
- A minibatch is an extra leading dimension over selected nuisance realizations.
- The cross matrix decides which channel relations STL is allowed to match.

Starting point: the object shape

In the full phase, both target and running objects have shape

$$X_{\text{target}}(I), \quad X_{\text{run}}(u; I) \in \mathbb{R}^{N_b \times C \times H \times W}.$$

For the run that crashes,

$$N_b = 15, \qquad C = 5, \qquad H = W = 384.$$

So one real batch tensor contains

$$15 \times 5 \times 384 \times 384 = 11{,}059{,}200$$

numbers.

The factor 8 below is because one `float64` number uses 8 bytes:

$$11{,}059{,}200 \text{ numbers} \times 8 \text{ bytes/number} \approx 88 \text{ MB} \quad (\approx 84 \text{ MiB}).$$

Two possible objectives

The difference is what statistics are requested from this same batch tensor:

$$\Phi(X) = \begin{cases} \Phi_{\mathrm{ST}}(X), & \texttt{ST\_COMPUTE\_PS=False}, \\ \big(\Phi_{\mathrm{ST}}(X),\, P(X)\big), & \texttt{ST\_COMPUTE\_PS=True}. \end{cases}$$

Here Φ_{ST} contains mean, variance, and scattering coefficients. The optional P contains cross-power-spectrum coefficients.

The final loss always compares target and running statistics:

$$\mathcal{L}(u; I) = \|\Phi(X_{\mathrm{run}}(u; I)) - \Phi(X_{\mathrm{target}}(I))\|_2^2.$$

Case 1: without the PS constraint

With

$$\texttt{ST\_COMPUTE\_PS=False},$$

the target side computes

$$y_{\rm target} = \bar{\Phi}_{\rm ST}(X_{\rm target}(I))\,,$$

and the running side computes

$$y_{\rm run}(u) = \bar{\Phi}_{\rm ST}(X_{\rm run}(u; I))\,.$$

The optimized loss is

$$\mathcal{L}_{\rm ST}(u; I) = \|y_{\rm run}(u) - y_{\rm target}\|_2^2.$$

Without PS: what is kept for gradients?

For the target:

$$y_{\mathrm{target}} = \bar{\Phi}_{\mathrm{ST}}(X_{\mathrm{target}}(I))$$

is computed inside `torch.no_grad()`, so it is just a fixed tensor.

For the running side, PyTorch keeps the graph needed for:

- the full running batch object $X_{\mathrm{run}}(u; I)$, not one selected nuisance draw,
- the wavelet/scattering intermediate tensors,
- the final flattened scattering vector $y_{\mathrm{run}}(u)$.

Here $I = \{i_1, \ldots, i_{N_b}\}$ contains all nuisance draws in the minibatch. So the first axis of X_{run} stores all N_b realizations at once.

No power-spectrum tensors are formed or stored for backward.

Without PS: memory scale

The full running object is

$$X_{\mathrm{run}}(u; I) \in \mathbb{R}^{15\times 5\times 384\times 384}.$$

Since it is real `float64`, each entry costs 8 bytes:

$$15 \times 5 \times 384 \times 384 \times 8 \approx 88 \text{ MB}.$$

If an intermediate representation of the same X is complex, each entry costs 16 bytes:

$$15 \times 5 \times 384 \times 384 \times 16 \approx 177 \text{ MB}.$$

The scattering transform then keeps wavelet/scattering intermediates for backward. Those are substantial, but they are tied to the scattering operator:

$$\text{roughly scales with } N_b,\ C,\ H,\ W,\ J,\ L.$$

Crucially, without PS there is no extra tensor with shape

$$N_b \times C \times C \times N_{\mathrm{bins}} \times H \times W.$$

Case 2: with the PS constraint

With

`ST_COMPUTE_PS=True`,

the target side computes two blocks:

$$y_{\text{target}} = \left(\bar{\Phi}_{\text{ST}}(X_{\text{target}}(I)), \bar{P}(X_{\text{target}}(I))\right).$$

The running side computes the matching two blocks:

$$y_{\text{run}}(u) = \left(\bar{\Phi}_{\text{ST}}(X_{\text{run}}(u; I)), \bar{P}(X_{\text{run}}(u; I))\right).$$

The loss is still one squared difference:

$$\mathcal{L}(u; I) = \|y_{\text{run}}(u) - y_{\text{target}}\|_2^2.$$

What is kept in memory with PS?

The target PS block is again fixed:

$$\bar{P}(X_{\text{target}}(I))$$

because it is computed under `torch.no_grad()`.

The running PS block is computed from the full running batch:

$$X_{\text{run}}(u; I) \in \mathbb{R}^{N_b \times C \times H \times W} \quad \longmapsto \quad \bar{P}(X_{\text{run}}(u; I)).$$

This block depends on u_Q, u_U, so PyTorch must keep its graph.

This does not mean one nuisance term is kept separately. The whole minibatch object $X_{\text{run}}(u; I)$ is the input to P.

In addition to the scattering graph, PyTorch keeps the PS intermediates needed so that

$$\frac{\partial \mathcal{L}_{\text{PS}}}{\partial u_Q}, \quad \frac{\partial \mathcal{L}_{\text{PS}}}{\partial u_U}$$

can be computed during `loss.backward()`.

With PS: the first large object

The FFT power-spectrum operator forms binned Fourier maps:

$$X_{\mathrm{bin}} \sim N_b \times C \times N_{\mathrm{bins}} \times H \times W.$$

This is still built from the full $X_{\mathrm{run}}(u; I)$. The N_b axis is the full minibatch of nuisance draws. For a 384 map, the default is

$$N_{\mathrm{bins}} = 24.$$

So, with complex dtype,

$$15 \times 5 \times 24 \times 384 \times 384 \times 16 \approx 4.25 \text{ GB} \quad (\approx 3.95 \text{ GiB}).$$

This is already much larger than the original real batch tensor.

With PS: the dominant cross object

The PS operator then forms channel-pair products of the binned X object:

$$X_{\text{cross}} \sim N_b \times C \times C \times N_{\text{bins}} \times H \times W.$$

The two C factors mean "channel against channel". This is why the memory scales like C^2, even though the final PS coefficients are later reduced.

For the current run:

$$15 \times 5 \times 5 \times 24 \times 384 \times 384 \times 16 \approx 21.23 \text{ GB} \quad (\approx 19.78 \text{ GiB}).$$

This number matches the failed allocation:

```
Tried to allocate 19.78 GiB.
```

So the OOM is not caused by the final number of PS coefficients. It is caused by forming this intermediate cross-product object from X.

With PS: what else is kept?

For non-periodic boundaries, the FFT PS path can also build intermediate objects from the same X:

- the binned Fourier maps X_{bin},
- inverse FFTs of those binned maps,
- crop masks,
- real-space products with the original X,
- the final reduced power-spectrum tensor $P(X)$.

The final $P(X)$ tensor is small compared to X_{cross}. The costly part is the temporary path used to get to $P(X)$.

During the running-side computation, PyTorch keeps enough of this path to compute

$$\frac{\partial \mathcal{L}_{\mathrm{PS}}}{\partial u_Q}, \qquad \frac{\partial \mathcal{L}_{\mathrm{PS}}}{\partial u_U}.$$

Why the coefficient count can look harmless

The printed line

`43706/361810 finite`

refers to the final flattened statistics after all reductions.

It does *not* count the temporary tensors used to compute those statistics:

$$X_{\mathrm{bin}}, \quad X_{\mathrm{cross}}, \quad \text{IFFT products}, \quad \text{saved backward tensors}.$$

Thus two runs can have similar final coefficient counts:

$$\dim y_{\mathrm{run}} \quad \text{similar},$$

but very different backward memory:

$$\text{scattering graph only} \quad \text{vs.} \quad \text{scattering graph} + \text{PS graph}.$$

The OOM occurs when the running PS graph is retained for differentiation.

Target side versus running side

For both settings, target statistics are computed under

$$\texttt{torch.no\_grad()}.$$

So the target PS computation may temporarily allocate large tensors, but it does not keep a backward graph.

The running side is different:

$$X_{\mathrm{run}}(u; I) \quad \text{depends on} \quad u_Q,\ u_U.$$

Therefore, with PS on, the large PS intermediates are saved for `loss.backward()`. That is why the crash happens after the coefficient count is printed.

Summary of the comparison

Without PS

- compute Φ_{ST},
- keep scattering intermediates for running side,
- no $P(X)$ tensors,
- no $N_b C^2 N_{\mathrm{bins}} HW$ object.

With PS

- compute Φ_{ST} and P,
- keep scattering intermediates,
- keep binned FFT maps,
- keep cross-spectrum object,
- about 19.78 GiB for one dominant tensor here.

The dominant PS memory scales as

$$N_b \, C^2 \, N_{\mathrm{bins}} \, H \, W.$$

Results

Loss curve is written to `results/loss_curve_planck.png`